Unified Agent Protocol (UAP)

A project specification for an AI-native control-plane protocol

Prepared for Dheeraj Kumar

June 10, 2026

Contents

1 Executive summary

Unified Agent Protocol (UAP) is a proposed AI-native control-plane protocol for building reliable, secure, observable, and efficient agentic applications. It is designed to sit above existing protocols rather than replace them. MCP, A2A, AG-UI, REST/OpenAPI, GraphQL, gRPC, AsyncAPI, CloudEvents, Kafka, and WebSockets remain valuable. UAP adds the missing coordination layer: intent, task lifecycle, capability selection, context budgeting, policy, identity delegation, provenance, structured error recovery, and traceable streaming events.

The core thesis is simple:

```
REST, gRPC, GraphQL, Kafka, and WebSockets move data.
MCP exposes tools and context.
A2A lets agents delegate work to agents.
AG-UI connects agents to user interfaces.
UAP coordinates all of them under one production-grade contract.
```

The protocol target is not the fastest wire format. The target is reducing the real bottlenecks in AI-native applications:

- sequential agent-tool loops;
- context-window/token bloat;
- tool catalog overload;
- unsafe or over-privileged tool invocation;
- fragmented identity and delegation;
- poor structured error recovery;
- weak provenance and auditability;
- hard-to-debug multi-protocol execution flows;
- lack of cost, latency, risk, and context budgets as first-class protocol data.

UAP's main design primitives are:

1. **Intent** - what the user or agent wants, separate from implementation.
2. **Task** - a durable unit of work with lifecycle, status, events, and artifacts.
3. **Capability** - a compact, typed, policy-aware description of what a tool, agent, workflow, or backend service can do.
4. **Context Contract** - an explicit budget and shape for information returned to the model.
5. **Policy and Provenance** - delegated identity, permission scope, approvals, safety gates, and audit trails.
6. **Execution Graph** - a task DAG that can be planned, optimized, parallelized, retried, and streamed.

UAP should begin as an adapter/gateway project and evolve into an open specification. The first implementation should be a TypeScript + Python reference runtime with adapters for MCP, OpenAPI, A2A, AG-UI, and AsyncAPI.

2 Research grounding

This proposal is grounded in current protocol specs and recent agent tooling research. The references below are summarized here because UAP should be compatible with the ecosystem rather than ignore it.

2.1 Current protocol landscape

MCP - Model Context Protocol. MCP standardizes how LLM applications connect to external tools, resources, prompts, and context. The current MCP lifecycle includes initialization, capability negotiation, normal operation, and shutdown; recent revisions include authorization, elicitation, sampling, structured tool outputs, and experimental task support. UAP should treat MCP servers as capability providers, not as the whole production control plane. [R1], [R2]

A2A - Agent2Agent Protocol. A2A focuses on interoperability between independent agent systems. Its goals include capability discovery, modality negotiation, collaborative task management, streaming, and asynchronous push notifications. UAP should map cross-agent delegation to A2A where an external agent is the right executor. [R3]

AG-UI - Agent User Interaction Protocol. AG-UI standardizes event-based, bidirectional interaction between agentic frontends and backends, including streaming chat, shared state, user interrupts, typed attachments, and frontend tool calls. UAP should expose events that can be projected into AG-UI for user interfaces. [R4]

ACP - Agent Communication Protocol. ACP's design emphasizes REST-native agent messaging, multimodality, synchronous/asynchronous communication, streaming, stateful/stateless modes, discovery, and long-running tasks. IBM now states ACP is part of A2A under the Linux Foundation direction. UAP should reuse lessons from ACP but avoid introducing another isolated agent-to-agent standard. [R5], [R6]

OpenAPI and AsyncAPI. OpenAPI removes guesswork in calling HTTP APIs by describing operations, parameters, schemas, and security. AsyncAPI provides a machine-readable description format for message-driven APIs and is explicitly protocol-agnostic across Kafka, WebSockets, AMQP, MQTT, STOMP, HTTP, and other transports. UAP should import these descriptions and wrap them in AI-native capability cards. [R7], [R8]

CloudEvents and trace context. CloudEvents standardizes event metadata for identification and routing. W3C Trace Context and OpenTelemetry context propagation let distributed systems correlate traces across network and process boundaries. UAP events should be CloudEvents-compatible and every task should carry trace context. [R9], [R10], [R11]

OAuth, DPoP, and DIDs. OAuth 2.1 continues the delegated authorization model for limited access to protected resources. DPoP sender-constrains OAuth tokens using proof-of-possession to reduce replay risk. W3C DIDs provide a standard identifier layer for decentralized identity. UAP should support existing enterprise identity through OAuth/OIDC and optionally support DIDs/verifiable credentials where organizations require portable agent identities. [R12], [R13], [R14]

2.2 Findings from recent research

Recent production and research work suggests that current agent protocols solve integration but not enough of the production problem.

A 2026 MCP production paper argues that three protocol-level primitives remain missing for production deployments: **identity propagation, adaptive tool budgeting, and structured error semantics**. This is directly reflected in UAP's identity envelope, budget block, and error model. [R15]

Security research on MCP highlights tool poisoning, prompt injection through metadata, client-side validation gaps, supply-chain risk, cross-system privilege escalation, and the need for provenance, scoped authorization, gateway policy, and sandboxing. UAP therefore treats capability metadata as untrusted until attested, scanned, policy-checked, and versioned. [R16], [R17], [R18]

Benchmarks such as MCPToolBench++ and MCP-Atlas show that real-world tool use is multi-domain, multi-step, heterogeneous, and limited by context-window pressure from long tool descriptions and schemas. UAP responds with compact capability cards, semantic indexes, context contracts, and task DAGs. [R19], [R20]

Recent systems work such as SUTRADHARA shows that agentic applications chain multiple LLM calls and tool executions before the final answer, creating a major latency bottleneck. It reports that tool calls can account for 30-80 percent of first-token-rendered latency and that sequential orchestration wastes potential parallelism. UAP therefore makes graph execution, streaming partial results, and budget-aware scheduling first-class. [R21]

3 Problem statement

AI-native systems are different from traditional apps. A normal web application usually calls a small number of APIs in response to explicit user actions. An agentic application may perform many dynamic steps:

```
user intent
  -> model reasoning
  -> tool discovery
  -> tool selection
  -> API call
  -> response inspection
  -> more reasoning
  -> second API call
  -> retrieval
  -> summarization
  -> action approval
  -> state mutation
  -> final answer
```

This creates bottlenecks that are not fully solved by REST, GraphQL, gRPC, MCP, or A2A alone.

3.1 Bottleneck 1: serial tool loops

The standard model tool-calling flow is a loop: model produces a tool call, application executes it, model observes result, and model decides whether to call another tool. This is simple but often slow. Multi-step workflows require many round trips and repeated model invocations.

UAP solution: represent work as a task graph that can be planned, optimized, parallelized, and retried by an execution runtime.

3.2 Bottleneck 2: context bloat

Agents often receive full API responses even when they need only a few fields. Every unnecessary token increases cost and latency, and it can distract the model.

UAP solution: make context budgets, field masks, output contracts, summarizers, rankers, and evidence requirements explicit in the protocol.

3.3 Bottleneck 3: tool overload

MCP-style tool discovery can expose hundreds or thousands of tools. Long natural language descriptions consume context and can confuse tool selection.

UAP solution: use compact capability cards with stable metadata, embeddings, risk labels, latency/cost estimates, and schema fingerprints. The model sees a small selected shortlist, not every raw tool descriptor.

3.4 Bottleneck 4: weak production identity propagation

An agent may act for a user, an organization, another agent, or an automated system. Current tool protocols often do not carry enough identity, delegation, resource, consent, and policy context throughout the workflow.

UAP solution: include actor, principal, subject, delegation chain, capability scope, token binding, approval state, and policy decision IDs in the envelope.

3.5 Bottleneck 5: unsafe side effects

A model can call tools that send emails, delete files, change records, transfer funds, or reveal sensitive data. Traditional API auth is necessary but not sufficient, because the risk comes from autonomous composition.

UAP solution: classify capabilities by risk, route high-risk actions through approval gates, enforce least privilege, attach provenance, and require signed side-effect receipts.

3.6 Bottleneck 6: poor error semantics

An agent cannot reliably recover from vague errors such as `500`, `failed`, or `invalid request`. It needs to know whether retrying is safe, whether parameters can be repaired, whether the issue is policy, latency, auth, rate limits, or missing user consent.

UAP solution: structured machine-readable errors with retry, repair, alternative capability, and user-action hints.

3.7 Bottleneck 7: observability gaps

A single agent answer may involve LLM calls, vector search, REST APIs, MCP servers, A2A tasks, queues, database queries, and user approvals. Without a shared task ID, trace context, event model, and provenance log, debugging is nearly impossible.

UAP solution: every request, event, artifact, tool result, and state mutation must carry a task ID, run ID, trace context, actor info, and provenance record.

4 Design goals and non-goals

4.1 Goals

UAP should:

1. provide a unified control plane for agentic workflows;
2. interoperate with MCP, A2A, AG-UI, OpenAPI, AsyncAPI, GraphQL, gRPC, and event systems;
3. reduce token, latency, and orchestration overhead;
4. make identity, delegation, policy, budgets, and provenance first-class;
5. support long-running, cancelable, resumable tasks;
6. support streaming partial results and state updates;
7. support human approval before risky actions;
8. standardize structured error recovery;
9. enable conformance testing and reference implementations;
10. be easy to adopt incrementally through gateways and adapters.

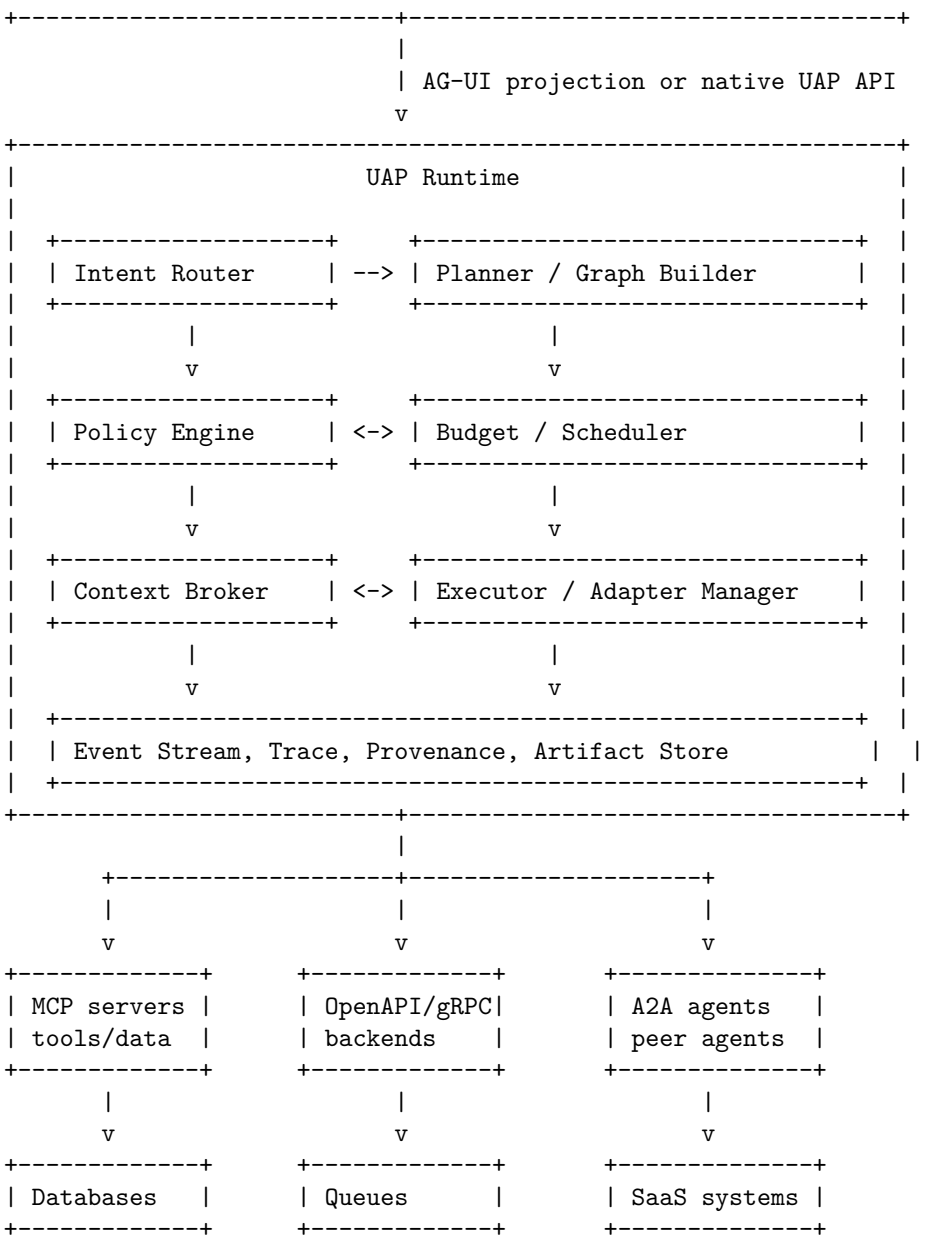
4.2 Non-goals

UAP should not:

1. replace HTTP, gRPC, WebSockets, Kafka, REST, GraphQL, MCP, A2A, or AG-UI;
2. require every backend service to be rewritten;
3. force one identity provider, LLM, model vendor, or agent framework;
4. expose raw chain-of-thought;
5. grant agents broad ambient authority;
6. assume that tool metadata is trusted;
7. make the LLM responsible for low-level retries, batching, joining, or deterministic data processing.

5 Conceptual architecture

```
+-------------------------------------------------------------+
|                        User / Client                        |
|            web app, IDE, mobile app, workflow UI            |
```



UAP is best understood as a control plane. It chooses, authorizes, schedules, and observes work. Existing APIs and event systems continue to do the actual transport and data movement.

6 Protocol roles

6.1 UAP Client

A UAP client initiates tasks, subscribes to events, submits approvals, receives artifacts, and may render UI state. A client can be:

- a browser app;
- an IDE;
- an enterprise workflow console;
- an agent framework;
- another UAP runtime;
- an A2A agent adapter;
- a background job.

6.2 UAP Runtime

The runtime receives intents and executes tasks. It owns:

- request validation;
- identity and delegation checks;
- capability registry lookup;
- planning and graph optimization;
- budget enforcement;
- context shaping;
- policy enforcement;
- adapter calls;
- event streaming;
- provenance logs;
- task persistence.

6.3 Capability Provider

A capability provider offers one or more executable capabilities. It can be:

- an MCP server;
- an OpenAPI-described REST service;
- a gRPC service;
- a GraphQL service;
- an A2A-compatible agent;
- a local function;
- a workflow engine;
- a vector database;
- a data warehouse;
- a queue or event stream.

6.4 Policy Authority

A policy authority decides whether an action is allowed. It may be a built-in engine, OPA/Rego, Cedar, Zanzibar-like authorization, cloud IAM, enterprise IAM, or a custom policy service.

6.5 Human Approver

A human approver is a user or administrator who must confirm a risky action, escalation, consent grant, or high-cost step.

7 Layer model

UAP can be described as seven layers.

```
L7  Application workflows
L6  Intent, planning, and task graph
L5  Policy, identity, delegation, consent
L4  Context contracts and artifact semantics
L3  Capability discovery and adapter mapping
L2  Event stream, lifecycle, errors, tracing
L1  Transports: HTTP, SSE, WebSocket, gRPC, queues
```

The same UAP message should be usable over multiple transports. The protocol semantics are independent of transport.

8 Transport bindings

UAP should define mandatory and optional bindings.

8.1 Mandatory binding: HTTPS + JSON

The minimum interoperable binding is HTTPS with JSON request/response.

Required endpoints:

```
GET  /.well-known/uap.json
GET  /uap/v1/capabilities
POST /uap/v1/tasks
GET  /uap/v1/tasks/{task_id}
POST /uap/v1/tasks/{task_id}/cancel
POST /uap/v1/tasks/{task_id}/approvals
GET  /uap/v1/tasks/{task_id}/events
GET  /uap/v1/artifacts/{artifact_id}
```

8.2 Streaming binding: SSE

SSE is the recommended simple streaming binding for browser and server-to-client updates.

```
GET /uap/v1/tasks/{task_id}/events?cursor=...
Accept: text/event-stream
```

8.3 Bidirectional binding: WebSocket

WebSocket is useful when the client must send interrupts, state diffs, approvals, or user input while receiving events.

```
GET /uap/v1/connect
Upgrade: websocket
```

8.4 High-performance binding: gRPC

gRPC is useful inside infrastructure boundaries for high-throughput task execution, typed SDKs, and bidirectional streaming.

8.5 Event binding: CloudEvents over broker

Long-running tasks and cross-service events may be published as CloudEvents over Kafka, NATS, AMQP, Pub/Sub, or another broker.

9 Discovery

Every UAP runtime exposes a discovery document:

```
{
  "uap_version": "0.1.0",
  "runtime_id": "did:web:runtime.example.com",
  "name": "Example UAP Runtime",
  "endpoints": {
    "tasks": "https://runtime.example.com/uap/v1/tasks",
    "capabilities": "https://runtime.example.com/uap/v1/capabilities",
    "events": "https://runtime.example.com/uap/v1/tasks/{task_id}/events"
  },
  "transports": ["https", "sse", "websocket"],
  "auth": {
    "schemes": ["oauth2", "dpop"],
    "authorization_server": "https://auth.example.com"
  },
  "features": {
    "task_graph": true,
    "approvals": true,
```

```
    "artifact_store": true,
    "context_budgeting": true,
    "policy_explain": true
  }
}
```

10 Core message envelope

Every UAP message uses a shared envelope. This avoids the common problem where identity, trace, budget, and policy disappear after the first API hop.

```
{
  "uap": "0.1.0",
  "message_id": "msg_01J...",
  "type": "task.invoke",
  "created_at": "2026-06-10T17:30:00Z",
  "trace": {
    "traceparent": "00-abcd...-0123...-01",
    "tracestate": "vendor=value"
  },
  "actor": {
    "agent_id": "did:web:agent.example",
    "principal_id": "user:123",
    "tenant_id": "org:asu",
    "session_id": "sess_456"
  },
  "delegation": {
    "mode": "on_behalf_of_user",
    "chain": [
      {
        "subject": "user:123",
        "delegate": "agent:sales-assistant",
        "scope": ["crm.read", "email.draft"],
        "expires_at": "2026-06-10T18:30:00Z"
      }
    ]
  },
  "budget": {
    "max_latency_ms": 8000,
    "max_cost_usd": 0.10,
    "max_context_tokens": 4000,
    "max_tool_calls": 12,
    "max_risk": "medium"
  },
  "policy": {
    "data_classes_allowed": ["public", "internal"],
    "approval_required_for": ["external_email.send"],
    "policy_set": "enterprise-default"
  },
  "body": {}
}
```

10.1 Envelope fields

`uap` is the protocol version.

`message_id` is unique and idempotency-safe.

`type` is the semantic event or command type.

`created_at` is the message timestamp.

`trace` carries W3C trace context.

`actor` identifies the principal, agent, tenant, and session.

`delegation` describes who delegated what to whom.

`budget` constrains latency, cost, context, calls, and risk.

`policy` carries policy hints and hard constraints.

`body` contains type-specific content.

11 Task lifecycle

A task is the durable unit of work.

```
created
  -> accepted
  -> planning
  -> waiting_for_approval
  -> running
  -> partially_completed
  -> completed

created
  -> rejected

running
  -> paused
  -> resumed
  -> canceled

running
  -> failed_recoverable
  -> retrying

running
  -> failed_terminal
```

11.1 Task creation

```
{
  "type": "task.invoke",
  "body": {
    "intent": {
      "goal": "Find overdue invoices and draft reminder emails.",
      "domain": "finance.operations",
      "success_criteria": [
        "Return top 10 overdue invoices",
        "Draft emails but do not send",
        "Include evidence for each invoice"
      ]
    },
    "inputs": {
      "customer_segment": "enterprise",
      "days_overdue_min": 15
```

```
    },
    "context_request": {
      "detail": "minimal",
      "fields": ["invoice_id", "customer", "amount", "due_date"],
      "max_items": 10,
      "evidence_required": true
    },
    "execution": {
      "mode": "server_optimized",
      "allow_parallelism": true,
      "allow_partial_results": true
    }
  }
}
```

11.2 Task response

```
{
  "task_id": "tsk_01J...",
  "status": "accepted",
  "event_stream": "/uap/v1/tasks/tsk_01J/events",
  "expires_at": "2026-06-11T17:30:00Z"
}
```

12 Capability model

UAP replaces raw tool lists with capability cards.

A capability can wrap:

- one MCP tool;
- a sequence of MCP tools;
- one OpenAPI operation;
- a GraphQL query or mutation;
- a gRPC method;
- a workflow;
- a vector search query;
- an A2A agent task;
- a human approval workflow.

12.1 Capability card

```
{
  "capability_id": "invoice.list_overdue.v1",
  "name": "List overdue invoices",
  "kind": "read",
  "provider": {
    "type": "openapi",
    "provider_id": "billing-api",
    "operation_id": "listOverdueInvoices"
  },
  "purpose": "Find overdue invoices matching filters.",
  "semantic_tags": ["finance", "invoice", "accounts_receivable"],
  "input_schema_ref": "schema:invoice.list_overdue.input.v1",
  "output_schema_ref": "schema:invoice.list_overdue.output.v1",
  "risk": {
    "level": "low",
    "side_effect": false,
```

```
    "data_classes": ["financial", "customer"]
  },
  "policy": {
    "required_scopes": ["invoice.read"],
    "approval": "not_required"
  },
  "performance": {
    "latency_p50_ms": 200,
    "latency_p95_ms": 1200,
    "cost_estimate_usd": 0.001
  },
  "context_cost": {
    "typical_tokens": 300,
    "max_tokens": 1500
  },
  "quality": {
    "success_rate_30d": 0.995,
    "schema_stability": "high",
    "last_verified_at": "2026-06-01T00:00:00Z"
  },
  "attestation": {
    "metadata_hash": "sha256:...",
    "signed_by": "did:web:registry.example.com",
    "signature": "..."
  }
}
```

12.2 Capability selection

The model should not receive all tools. The runtime should:

1. parse intent;
2. retrieve candidate capabilities by semantic index;
3. filter by policy, tenant, identity, and budget;
4. rank by fit, risk, latency, cost, freshness, and reliability;
5. send only a short candidate set to the planner or model.

Candidate selection output:

```
{
  "intent_id": "int_123",
  "candidates": [
    {
      "capability_id": "invoice.list_overdue.v1",
      "score": 0.94,
      "why": "Matches overdue invoice retrieval with required fields.",
      "risk": "low"
    },
    {
      "capability_id": "email.draft.v2",
      "score": 0.89,
      "why": "Can draft but not send external email.",
      "risk": "medium"
    }
  ]
}
```

13 Context contract

The context contract tells providers and adapters what information is useful to return to the model.

```
{
  "context_request": {
    "max_context_tokens": 3000,
    "detail": "minimal",
    "fields": ["id", "status", "amount", "due_date", "source_url"],
    "max_items": 10,
    "sort": [{"field": "due_date", "order": "asc"}],
    "evidence_required": true,
    "redaction": {
      "pii": "mask",
      "secrets": "drop"
    },
    "summarization": {
      "allowed": true,
      "mode": "extractive_with_citations",
      "include_confidence": true
    }
  }
}
```

13.1 Context modes

`raw` returns raw data and should be rare.

`minimal` returns only the fields required by the task.

`summary` returns a structured summary with source pointers.

`evidence` returns compact claims plus provenance.

`artifact` returns files, tables, images, or records outside the model context with short handles inside the context.

13.2 Context shaping pipeline

```
provider response
  -> schema validation
  -> security scan
  -> field mask
  -> policy filter
  -> ranking
  -> deduplication
  -> redaction
  -> summarization
  -> provenance attachment
  -> token budget check
  -> model-visible context
```

14 Planning and task graphs

UAP represents complex work as a graph. A graph node can be a capability call, planner call, transform, retrieval, approval, human input request, artifact creation, or external agent delegation.

14.1 Graph node

```
{
  "node_id": "n2",
```

```
  "kind": "capability.call",
  "capability_id": "invoice.list_overdue.v1",
  "depends_on": ["n1"],
  "input": {
    "days_overdue_min": 15
  },
  "budget": {
    "timeout_ms": 2000,
    "max_retries": 2
  },
  "on_error": {
    "strategy": "fallback",
    "fallback_capability_id": "invoice.search_cached.v1"
  }
}
```

14.2 Graph example

```
{
  "type": "task.plan.proposed",
  "body": {
    "graph": {
      "nodes": [
        {
          "node_id": "n1",
          "kind": "capability.call",
          "capability_id": "customer.segment.resolve.v1"
        },
        {
          "node_id": "n2",
          "kind": "capability.call",
          "capability_id": "invoice.list_overdue.v1",
          "depends_on": ["n1"]
        },
        {
          "node_id": "n3",
          "kind": "transform.context_shape",
          "depends_on": ["n2"]
        },
        {
          "node_id": "n4",
          "kind": "capability.call",
          "capability_id": "email.draft.v2",
          "depends_on": ["n3"]
        },
        {
          "node_id": "n5",
          "kind": "approval.request",
          "depends_on": ["n4"],
          "approval_type": "review_drafts"
        }
      ]
    }
  }
}
```

14.3 Execution modes

`client_planned`: client provides the graph.

`model_planned`: model proposes the graph; runtime validates and executes.

`server_optimized`: runtime builds and optimizes the graph.

`hybrid`: model proposes, runtime rewrites for safety/performance.

14.4 Scheduling rules

The runtime should:

- execute independent read nodes in parallel;
- avoid parallelizing conflicting writes unless explicitly safe;
- enforce idempotency keys for side effects;
- apply budget allocation per node;
- use cached results when allowed;
- stream partial results when useful;
- pause at approval nodes;
- persist node outputs and provenance.

15 Event model

UAP events should be CloudEvents-compatible.

```
{
  "specversion": "1.0",
  "type": "uap.task.node.completed",
  "source": "uap://runtime/example",
  "id": "evt_01J...",
  "time": "2026-06-10T17:30:03Z",
  "subject": "tsk_01J/n2",
  "datacontenttype": "application/json",
  "traceparent": "00-abcd...-0123...-01",
  "data": {
    "task_id": "tsk_01J",
    "node_id": "n2",
    "status": "completed",
    "summary": "Found 8 overdue invoices.",
    "artifact_ids": ["art_invoice_table_123"]
  }
}
```

15.1 Standard event types

Task events:

```
uap.task.created
uap.task.accepted
uap.task.rejected
uap.task.planning.started
uap.task.plan.proposed
uap.task.plan.accepted
uap.task.running
uap.task.paused
uap.task.resumed
uap.task.completed
```

`uap.task.canceled`
`uap.task.failed`

Node events:

`uap.node.started`
`uap.node.progress`
`uap.node.completed`
`uap.node.failed`
`uap.node.retrying`
`uap.node.skipped`

Approval events:

`uap.approval.requested`
`uap.approval.granted`
`uap.approval.denied`
`uap.approval.expired`

Context and artifact events:

`uap.context.ready`
`uap.context.truncated`
`uap.artifact.created`
`uap.artifact.updated`
`uap.artifact.redacted`

Policy and security events:

`uap.policy.allowed`
`uap.policy.denied`
`uap.policy.escalated`
`uap.security.warning`
`uap.security.blocked`

16 Structured errors

UAP errors must be designed for deterministic recovery.

```
{
  "type": "uap.error",
  "code": "RATE_LIMITED",
  "message": "Provider rate limit exceeded.",
  "category": "transient",
  "recoverability": "retryable",
  "safe_retry": true,
  "retry_after_ms": 2500,
  "affected_node": "n2",
  "alternatives": [
    {
      "capability_id": "invoice.search_cached.v1",
      "tradeoff": "May be up to 15 minutes stale."
    }
  ],
  "repair_hint": {
    "action": "wait_or_use_fallback"
  },
  "user_action_required": false
}
```

16.1 Error categories

```
invalid_input
schema_mismatch
auth_required
consent_required
policy_denied
approval_required
rate_limited
timeout
provider_unavailable
context_budget_exceeded
cost_budget_exceeded
unsafe_output
conflict
non_idempotent_retry_blocked
unknown
```

16.2 Recoverability values

```
retryable
repairable_by_model
repairable_by_user
requires_approval
requires_auth
fallback_available
terminal
```

17 Identity, delegation, and authorization

UAP should not invent identity. It should integrate with existing systems.

Recommended default:

- OIDC for authentication;
- OAuth 2.1 style delegated authorization;
- OAuth resource indicators for resource-bound tokens;
- DPoP or mTLS for sender-constrained tokens where replay risk matters;
- enterprise IAM for internal authorization;
- optional DID/verifiable credential identities for cross-organization agents.

17.1 Actor model

```
{
  "actor": {
    "principal_id": "user:123",
    "principal_type": "human",
    "agent_id": "agent:finance-assistant",
    "agent_instance_id": "agent-runner:abc",
    "tenant_id": "org:asu",
    "session_id": "sess_456"
  }
}
```

17.2 Delegation chain

```
{
  "delegation": {
```

```
    "chain": [
      {
        "issuer": "user:123",
        "subject": "agent:finance-assistant",
        "audience": "uap://runtime/finance",
        "scope": ["invoice.read", "email.draft"],
        "constraints": {
          "max_cost_usd": 0.10,
          "expires_at": "2026-06-10T18:00:00Z",
          "no_external_send": true
        },
        "proof": {
          "type": "jwt",
          "token_hash": "sha256:..."
        }
      }
    ]
  }
}
```

17.3 Authorization rule

Every capability invocation must satisfy:

```
principal is authenticated
AND agent is authorized to act for principal
AND requested capability is in delegated scope
AND resource is allowed by tenant policy
AND data class is allowed
AND risk is within budget
AND required approval exists for side effects
AND token is valid for target resource
```

18 Policy and approvals

Policies should be machine-enforceable and explainable.

18.1 Risk levels

```
none       informational only
low        read-only, internal data
medium     drafting, internal writes, sensitive read
high       external communication, financial action, destructive write
critical   legal, payment, production infra, irreversible operation
```

18.2 Approval request

```
{
  "type": "uap.approval.requested",
  "body": {
    "approval_id": "apr_123",
    "task_id": "tsk_01J",
    "reason": "External email send requires human approval.",
    "requested_action": {
      "capability_id": "email.send.v1",
      "recipient_count": 8,
      "data_classes": ["customer", "financial"]
```

```
    },
    "preview_artifacts": ["art_email_drafts_456"],
    "options": ["approve", "deny", "edit", "approve_subset"],
    "expires_at": "2026-06-10T18:00:00Z"
  }
}
```

18.3 Approval response

```
{
  "approval_id": "apr_123",
  "decision": "approve_subset",
  "approved_items": ["draft_1", "draft_2"],
  "comment": "Send only to customers with more than 30 days overdue."
}
```

19 Provenance and audit

Every model-visible claim and side effect should be traceable.

19.1 Provenance record

```
{
  "provenance_id": "prov_123",
  "task_id": "tsk_01J",
  "node_id": "n2",
  "capability_id": "invoice.list_overdue.v1",
  "provider": "billing-api",
  "input_hash": "sha256:...",
  "output_hash": "sha256:...",
  "source_refs": [
    {
      "type": "record",
      "system": "billing-db",
      "record_id": "inv_456",
      "version": "2026-06-10T00:00:00Z"
    }
  ],
  "policy_decision_id": "pdp_789",
  "actor": {
    "principal_id": "user:123",
    "agent_id": "agent:finance-assistant"
  },
  "created_at": "2026-06-10T17:30:03Z"
}
```

19.2 Audit requirements

UAP compliant runtimes must store:

- task creation request;
- normalized plan;
- capability versions used;
- policy decisions;
- approval decisions;
- inputs and outputs or hashes where storing raw data is prohibited;
- generated artifacts;

- side-effect receipts;
- trace IDs;
- final response metadata.

20 Adapter mappings

UAP succeeds only if it works with existing systems.

20.1 MCP adapter

Mapping:

```
MCP server              -> UAP capability provider
MCP tool                -> UAP capability
MCP resource            -> UAP context source or artifact
MCP prompt              -> UAP prompt template capability
MCP task                -> UAP task or delegated node
MCP structured output   -> UAP output schema
MCP elicitation         -> UAP approval or user-input request
```

Adapter duties:

- fetch MCP tools and resources;
- verify and sanitize tool metadata;
- generate capability cards;
- add schema fingerprints;
- enforce policy before tool invocation;
- shape tool output according to the context contract;
- attach provenance;
- map MCP errors to UAP errors.

20.2 A2A adapter

Mapping:

```
A2A agent card       -> UAP capability provider card
A2A capability       -> UAP capability
A2A task             -> UAP delegated task node
A2A streaming update -> UAP event
A2A artifact         -> UAP artifact
```

Adapter duties:

- discover agent cards;
- verify agent identity and transport security;
- translate UAP tasks into A2A tasks;
- map A2A status updates into UAP events;
- enforce delegation scope across agent boundaries.

20.3 AG-UI adapter

Mapping:

```
UAP task events       -> AG-UI run events
UAP approval request  -> AG-UI interrupt/input event
UAP artifacts         -> AG-UI typed attachments or UI components
UAP progress          -> AG-UI state diffs
UAP final answer      -> AG-UI message snapshot
```

Adapter duties:

- keep frontend state synchronized;
- support cancel/resume;
- present approvals;
- avoid exposing raw chain-of-thought;
- show provenance and traceable steps in a user-friendly way.

20.4 OpenAPI adapter

Mapping:

```
OpenAPI document      -> UAP provider catalog
operationId           -> UAP capability
parameters/requestBody -> UAP input schema
responses             -> UAP output schema
security schemes      -> UAP auth requirements
```

Adapter duties:

- import OpenAPI 3.x specs;
- infer risk and data classes from extensions;
- support vendor extensions such as `x-uap-risk` and `x-uap-context-cost`;
- convert HTTP errors into UAP errors;
- support idempotency keys for side effects.

Example extension:

```yaml
x-uap:
  capability_id: invoice.list_overdue.v1
  risk:
    level: low
    side_effect: false
    data_classes: [financial, customer]
  context_cost:
    typical_tokens: 300
    max_tokens: 1500
```

20.5 AsyncAPI and event adapter

Mapping:

```
AsyncAPI channel      -> UAP event source or sink
AsyncAPI operation    -> UAP capability
message schema        -> UAP input/output schema
broker binding        -> UAP transport binding
CloudEvents metadata  -> UAP event envelope
```

Adapter duties:

- subscribe to event-driven state changes;
- emit task lifecycle events;
- support asynchronous callbacks;
- preserve causal ordering where required;
- handle idempotent event consumption.

20.6 GraphQL adapter

Mapping:

```
GraphQL query     -> UAP read capability
GraphQL mutation  -> UAP write capability
schema            -> UAP schemas and capability metadata
selection set     -> UAP context field mask
```

Adapter duties:

- generate minimal selection sets from context contracts;
- enforce query depth and cost limits;
- prevent N+1 backend explosions where possible;
- map GraphQL errors to UAP structured errors.

20.7 gRPC adapter

Mapping:

```
protobuf service -> UAP provider
rpc method       -> UAP capability
proto messages   -> UAP schemas
metadata         -> UAP headers/envelope fields
streaming rpc    -> UAP event or artifact stream
```

Adapter duties:

- propagate identity and trace metadata;
- stream partial results;
- validate protobuf schemas;
- map status codes to UAP errors.

21 Runtime components

21.1 1. API gateway

Responsibilities:

- TLS termination;
- auth verification;
- rate limiting;
- request size limits;
- idempotency;
- schema validation;
- tenant routing;
- admission control.

21.2 2. Intent router

Responsibilities:

- normalize the user or agent goal;
- classify domain and risk;
- retrieve candidate capabilities;
- decide whether the task can be handled locally or delegated.

21.3 3. Capability registry

Responsibilities:

- store capability cards;
- index semantic embeddings;
- maintain schema fingerprints;
- track provider health;
- store attestation and version history;
- expose discovery APIs.

21.4 4. Planner

Responsibilities:

- build task graphs;
- validate graph safety;
- estimate cost/latency/context;
- identify approval nodes;
- choose fallback capabilities;
- produce explainable plans.

21.5 5. Budget scheduler

Responsibilities:

- allocate latency budget across nodes;
- allocate tool-call budget;
- enforce token budget;
- schedule parallel calls;
- stop runaway loops;
- downgrade detail if budget is low.

21.6 6. Policy engine

Responsibilities:

- evaluate capability invocation;
- enforce data-class restrictions;
- require approval for risky actions;
- validate delegation chain;
- produce policy decision records;
- explain denials in machine-readable form.

21.7 7. Context broker

Responsibilities:

- retrieve and shape data;
- apply field masks;
- rank and deduplicate;
- redact sensitive data;
- summarize with citations/provenance;
- create artifacts for large outputs;
- enforce token limits.

21.8 8. Executor

Responsibilities:

- run graph nodes;
- call adapters;
- handle retries;
- stream events;
- checkpoint state;
- cancel/resume/pause tasks;
- produce receipts.

21.9 9. Artifact store

Responsibilities:

- store large files, tables, images, logs, drafts, and records;
- provide signed URLs or scoped access handles;
- attach provenance and retention policies;
- support redaction and expiry.

21.10 10. Observability service

Responsibilities:

- emit traces, metrics, and logs;
- correlate UAP task IDs with backend spans;
- expose task timelines;
- support replay in safe environments;
- produce compliance audit exports.

22 Reference API specification

22.1 Create task

```
POST /uap/v1/tasks
Authorization: DPoP <access_token>
Content-Type: application/json
Idempotency-Key: idem_123
```

Response:

```
{
  "task_id": "tsk_01J...",
  "status": "accepted",
  "event_stream": "/uap/v1/tasks/tsk_01J/events",
  "links": {
    "self": "/uap/v1/tasks/tsk_01J",
    "cancel": "/uap/v1/tasks/tsk_01J/cancel"
  }
}
```

22.2 Get task

```
GET /uap/v1/tasks/{task_id}
```

Response:

```
{
  "task_id": "tsk_01J...",
  "status": "waiting_for_approval",
  "progress": {
    "completed_nodes": 3,
    "total_nodes": 5
  },
  "current_approval_id": "apr_123",
  "artifacts": ["art_email_drafts_456"]
}
```

22.3 Subscribe to events

```
GET /uap/v1/tasks/{task_id}/events?cursor=evt_100
Accept: text/event-stream
```

SSE event:

```
event: uap.node.completed
id: evt_101
data: {"task_id":"tsk_01J","node_id":"n2","status":"completed"}
```

22.4 Submit approval

```
POST /uap/v1/tasks/{task_id}/approvals
Content-Type: application/json
```

```
{
  "approval_id": "apr_123",
  "decision": "approve",
  "comment": "Looks good."
}
```

22.5 Cancel task

```
POST /uap/v1/tasks/{task_id}/cancel
```

```
{
  "reason": "User canceled from UI."
}
```

23 SDK design

The reference SDK should be small and boring. It should hide transport details but expose task events, approvals, artifacts, and typed schemas.

23.1 TypeScript example

```
import { UapClient } from "@uap/sdk";

const client = new UapClient({
  baseUrl: "https://runtime.example.com",
  auth: myAuthProvider
});

const task = await client.tasks.create({
  intent: {
    goal: "Find overdue invoices and draft reminders",
    domain: "finance.operations"
  },
  contextRequest: {
    detail: "minimal",
    maxItems: 10,
    evidenceRequired: true
  },
  budget: {
    maxLatencyMs: 8000,
    maxContextTokens: 3000
  }
});

for await (const event of client.tasks.events(task.taskId)) {
  if (event.type === "uap.approval.requested") {
    await client.tasks.approve(task.taskId, {
      approvalId: event.data.approvalId,
      decision: "approve"
```

```
    });
  }
  if (event.type === "uap.task.completed") {
    console.log(event.data.result);
  }
}
```

23.2 Python example

```
from uap import UAPClient

client = UAPClient(
    base_url="https://runtime.example.com",
    auth=my_auth_provider,
)

task = client.tasks.create(
    intent={
        "goal": "Find overdue invoices and draft reminders",
        "domain": "finance.operations",
    },
    context_request={
        "detail": "minimal",
        "max_items": 10,
        "evidence_required": True,
    },
    budget={
        "max_latency_ms": 8000,
        "max_context_tokens": 3000,
    },
)

for event in client.tasks.events(task.task_id):
    if event.type == "uap.approval.requested":
        client.tasks.approve(
            task.task_id,
            approval_id=event.data["approval_id"],
            decision="approve",
        )
    if event.type == "uap.task.completed":
        print(event.data["result"])
```

24 Reference implementation plan

24.1 Repository structure

```
uap/
  specs/
    uap-core.md
    uap-events.md
    uap-capability-card.schema.json
    uap-task.schema.json
    uap-error.schema.json
    uap-openapi-extensions.md
  packages/
    typescript-sdk/
    python-sdk/
```

```
    conformance-tests/
    registry/
    runtime-core/
    adapter-mcp/
    adapter-openapi/
    adapter-a2a/
    adapter-ag-ui/
    adapter-asyncapi/
    adapter-graphql/
    adapter-grpc/
  examples/
    invoice-agent/
    customer-support-agent/
    research-agent/
    devops-agent/
  deployments/
    docker-compose/
    helm-chart/
    terraform/
  docs/
    quickstart.md
    security.md
    adapter-authoring.md
    operations.md
```

24.2 Milestone 0: specification skeleton

Deliverables:

- core envelope schema;
- task lifecycle specification;
- capability card schema;
- context contract schema;
- error schema;
- event type registry;
- security model draft;
- conformance requirements.

Acceptance criteria:

- JSON Schemas validate sample messages;
- all standard event types are documented;
- every required field has a clear meaning;
- examples run through schema validators.

24.3 Milestone 1: minimal runtime

Deliverables:

- HTTPS API;
- task persistence;
- SSE event stream;
- local function capability adapter;
- capability registry;
- basic policy engine;
- artifact store;
- TypeScript and Python SDKs.

Acceptance criteria:

- create task;
- execute task graph with 2-3 local capabilities;
- stream events;
- create artifact;
- cancel task;
- emit trace ID;
- pass conformance tests.

24.4 Milestone 2: MCP and OpenAPI adapters

Deliverables:

- MCP server discovery;
- MCP tool to UAP capability conversion;
- OpenAPI import;
- schema fingerprinting;
- structured error mapping;
- basic metadata scanner;
- context shaping.

Acceptance criteria:

- import an MCP server and call a tool through UAP;
- import an OpenAPI spec and call operations through UAP;
- tool output is shaped by context contract;
- unsafe metadata is flagged;
- policy can deny high-risk tools.

24.5 Milestone 3: policy, approvals, and provenance

Deliverables:

- approval events;
- human approval API;
- signed side-effect receipts;
- provenance record store;
- audit export;
- OIDC/OAuth integration;
- DPoP support for high-risk calls.

Acceptance criteria:

- high-risk operation pauses for approval;
- approval resumes graph;
- denial stops graph safely;
- audit export shows who/what/why/when;
- provenance links final claims to sources.

24.6 Milestone 4: A2A and AG-UI adapters

Deliverables:

- A2A agent card import;
- delegated A2A task nodes;
- AG-UI event projection;
- frontend approval widgets;
- cancel/resume from UI.

Acceptance criteria:

- UAP delegates one graph node to an A2A agent;
- task updates stream into an AG-UI frontend;

- user approval is submitted from the frontend;
- raw chain-of-thought is never exposed.

24.7 Milestone 5: production hardening

Deliverables:

- Kubernetes deployment;
- Postgres persistence;
- Redis/NATS/Kafka event backend option;
- OpenTelemetry tracing;
- rate limiting;
- policy engine plugins;
- tenant isolation;
- secret management;
- load tests;
- security review.

Acceptance criteria:

- 99.9 percent control-plane API availability under target load;
- p95 task creation latency below 200 ms excluding execution;
- p95 event delivery latency below 500 ms;
- deterministic replay for test tasks;
- passing security baseline.

25 Implementation details

25.1 Storage model

Minimum tables or collections:

```
tasks
  task_id
  tenant_id
  principal_id
  agent_id
  status
  intent_json
  budget_json
  policy_json
  created_at
  updated_at

nodes
  node_id
  task_id
  capability_id
  status
  input_json
  output_ref
  retry_count
  started_at
  completed_at

events
  event_id
  task_id
  type
```

```
  data_json
  traceparent
  created_at

capabilities
  capability_id
  version
  provider_id
  card_json
  metadata_hash
  embedding_ref
  status
  updated_at

artifacts
  artifact_id
  task_id
  type
  uri
  content_hash
  data_class
  retention_policy
  created_at

policy_decisions
  decision_id
  task_id
  node_id
  decision
  reason_json
  policy_version
  created_at

approvals
  approval_id
  task_id
  status
  requested_action_json
  decision_json
  approver_id
  created_at
  decided_at

provenance
  provenance_id
  task_id
  node_id
  source_refs_json
  input_hash
  output_hash
  policy_decision_id
  created_at
```

25.2 Idempotency

Every state-changing command should accept an idempotency key. Side-effecting capabilities must expose either:

- an idempotency key field;

- a provider transaction ID;
- a safe dry-run/preview mode;
- or a runtime-managed side-effect receipt.

Non-idempotent actions must not be retried automatically unless the provider returns a receipt proving the operation did not occur.

25.3 Caching

Cache layers:

- capability cards and embeddings;
- schema imports;
- read-only provider results;
- shaped context outputs;
- artifact previews;
- policy decisions when allowed.

Cache entries must include:

- tenant;
- principal or delegation scope if user-specific;
- provider version;
- data classification;
- freshness TTL;
- provenance references.

25.4 Context compression

Recommended strategy:

1. Prefer field masks over summarization.
2. Prefer structured rows over prose.
3. Prefer artifact handles over large inline payloads.
4. Prefer extractive summaries with source references.
5. Include confidence and missing-data indicators.
6. Never summarize secrets or credentials into model-visible context.

25.5 Tool metadata scanning

Capability metadata should be scanned before indexing. Checks include:

- hidden instructions in descriptions;
- attempts to override system or developer instructions;
- instructions to exfiltrate secrets;
- high-risk behavior hidden under low-risk labels;
- schema-description mismatch;
- overly broad input fields;
- suspicious URLs;
- untrusted provider signatures;
- metadata drift from previous versions.

Scanner output:

```
{
  "capability_id": "file.read.v1",
  "metadata_hash": "sha256:...",
  "verdict": "quarantine",
  "risk_findings": [
    {
      "code": "HIDDEN_PROMPT_INJECTION",
```

```
      "severity": "high",
      "field": "description"
    }
  ]
}
```

26 Conformance profiles

UAP should define multiple conformance levels.

26.1 UAP-Core

A runtime supports:

- discovery;
- task creation;
- task status;
- task events;
- capability listing;
- structured errors;
- trace IDs;
- JSON Schema validation.

26.2 UAP-Secure

Adds:

- OAuth/OIDC auth;
- delegated actor envelope;
- policy enforcement;
- approval workflow;
- provenance records;
- audit export;
- metadata scanning.

26.3 UAP-Graph

Adds:

- task DAGs;
- parallel execution;
- retries and fallbacks;
- budget scheduling;
- checkpoint/resume.

26.4 UAP-Adapter

Adds at least two certified adapters:

- MCP;
- OpenAPI;
- A2A;
- AG-UI;
- AsyncAPI;
- GraphQL;
- gRPC.

26.5 UAP-Enterprise

Adds:

- tenant isolation;
- policy plugins;
- DPoP or mTLS;
- OpenTelemetry;
- high-availability deployment;
- retention controls;
- compliance audit exports;
- conformance test suite.

27 Evaluation plan

UAP should be evaluated against measurable bottlenecks.

27.1 Latency metrics

- first event latency;
- first useful result latency;
- final answer latency;
- task graph scheduling overhead;
- adapter call latency;
- approval wait time;
- event delivery latency.

27.2 Token and context metrics

- raw provider tokens avoided;
- model-visible context tokens;
- number of capability descriptions shown to model;
- context truncation rate;
- answer accuracy with reduced context.

27.3 Tool-use metrics

- number of tool calls per task;
- sequential depth;
- parallelism ratio;
- retry count;
- fallback count;
- tool selection accuracy;
- tool parameter repair rate.

27.4 Safety metrics

- policy denials;
- approval escalations;
- blocked high-risk actions;
- metadata poisoning detections;
- cross-tenant access attempts;
- audit completeness.

27.5 Reliability metrics

- task completion rate;
- partial completion rate;

- recoverable error success rate;
- terminal failure rate;
- duplicate side-effect rate;
- replay determinism.

27.6 Suggested benchmark tasks

1. Customer support: retrieve customer account, summarize tickets, draft reply.
2. Finance: find overdue invoices, draft reminders, wait for approval.
3. DevOps: inspect alert, query logs, propose remediation, require approval.
4. Research: search internal docs, produce cited summary, create artifact.
5. Procurement: compare vendors, create draft purchase request.
6. Multi-agent: delegate specialized analysis to an external A2A agent.

28 Security threat model

28.1 Assets

- user data;
- enterprise data;
- credentials and tokens;
- capability metadata;
- task plans;
- artifacts;
- audit logs;
- side-effect authority;
- model-visible context.

28.2 Adversaries

- malicious user;
- compromised MCP server;
- malicious tool metadata author;
- compromised provider API;
- over-privileged agent;
- prompt-injection content source;
- cross-tenant attacker;
- supply-chain attacker;
- network attacker;
- rogue insider.

28.3 Threats and mitigations

28.3.1 Tool poisoning

Threat: malicious instructions are embedded in tool metadata or schemas.

Mitigations:

- signed capability cards;
- metadata scanning;
- trusted registry allowlists;
- human-readable diffs on metadata changes;
- model-visible descriptor minimization;
- parameter visibility;
- capability risk labels;
- policy enforcement outside the model.

28.3.2 Prompt injection through tool output

Threat: retrieved content instructs the model to ignore rules or exfiltrate data.

Mitigations:

- content isolation;
- source labeling;
- retrieval sanitization;
- instruction/data separation;
- output policy checks;
- sensitive action approvals;
- no secrets in model-visible context.

28.3.3 Cross-system privilege escalation

Threat: low-risk read from one system combines with high-risk write in another.

Mitigations:

- delegation chain checks;
- least-privilege scopes;
- policy over whole graph, not just individual calls;
- approval for cross-boundary writes;
- data-class taint tracking.

28.3.4 Replay or token theft

Threat: a captured token is reused.

Mitigations:

- DPoP or mTLS for high-risk operations;
- short-lived tokens;
- resource-bound tokens;
- token audience validation;
- per-task scoped credentials.

28.3.5 Non-idempotent retries

Threat: retrying a failed action repeats a side effect.

Mitigations:

- idempotency keys;
- side-effect receipts;
- no automatic retry unless safe;
- provider reconciliation before retry.

28.3.6 Audit tampering

Threat: malicious actor hides or modifies evidence.

Mitigations:

- append-only logs;
- hash chains;
- external log sinks;
- signed receipts;
- retention policies;
- separation of duties.

29 Governance model

UAP should be developed as an open specification with vendor-neutral governance.

Recommended structure:

- public specification repository;
- semantic versioning;
- RFC process for major changes;
- reference implementation;
- conformance test suite;
- security working group;
- adapter working group;
- registry working group;
- interoperability events.

Compatibility policy:

- minor versions must be backward compatible;
- major versions can break schemas but must include migration guidance;
- experimental features must be namespaced;
- extension fields use `x-uap-*` for external specs and `extensions` in UAP JSON.

30 Adoption strategy

30.1 Phase 1: UAP as internal gateway

Start with one organization, one runtime, and a few adapters.

```
app -> UAP runtime -> MCP/OpenAPI/internal APIs
```

Use cases:

- support assistant;
- finance operations assistant;
- internal research assistant;
- DevOps assistant.

30.2 Phase 2: UAP for policy and observability

Add:

- approvals;
- provenance;
- OpenTelemetry;
- centralized capability registry;
- risk dashboards;
- metadata scanning.

30.3 Phase 3: UAP for graph optimization

Add:

- task DAG planning;
- parallel execution;
- retries and fallbacks;
- adaptive context budgets;
- caching;
- multi-provider routing.

30.4 Phase 4: UAP ecosystem interoperability

Add:

- A2A adapter;
- AG-UI projection;
- AsyncAPI event integration;
- third-party capability providers;
- conformance tests.

31 Example end-to-end flow

User request:

```
Find enterprise customers with invoices more than 30 days overdue,
prepare reminder emails, and show me before sending anything.
```

UAP runtime flow:

```
1. Create task.
2. Authenticate user and agent.
3. Retrieve candidate capabilities.
4. Filter by policy and budget.
5. Build graph:
   a. resolve customer segment;
   b. list overdue invoices;
   c. retrieve contact preferences;
   d. draft emails;
   e. request approval.
6. Execute read nodes in parallel where possible.
7. Shape context to top records with evidence.
8. Draft emails as artifacts.
9. Emit approval request.
10. User reviews drafts.
11. If approved, send emails or stop at draft state.
12. Write audit and provenance records.
```

Final response shape:

```
{
  "task_id": "tsk_01J...",
  "status": "completed",
  "summary": "Found 8 enterprise customers with invoices over 30 days overdue.",
  "artifacts": [
    {
      "artifact_id": "art_invoice_table_123",
      "type": "table",
      "title": "Overdue invoices"
    },
    {
      "artifact_id": "art_email_drafts_456",
      "type": "draft_set",
      "title": "Reminder email drafts"
    }
  ],
  "provenance": ["prov_123", "prov_124"]
}
```

32 Minimum viable protocol spec

The MVP should include only what is required to prove the concept.

32.1 Required objects

1. Discovery document.
2. Capability card.
3. Task invoke request.
4. Task status response.
5. Event envelope.
6. Error object.
7. Context request.
8. Approval request/response.
9. Artifact reference.
10. Provenance record.

32.2 Required APIs

1. List capabilities.
2. Create task.
3. Get task.
4. Subscribe to events.
5. Submit approval.
6. Cancel task.
7. Get artifact.

32.3 Required runtime behavior

1. Validate JSON.
2. Enforce identity and scope.
3. Enforce budget.
4. Emit standard events.
5. Pause on approvals.
6. Produce structured errors.
7. Store provenance.
8. Never expose raw chain-of-thought.

33 Appendix A: Minimal JSON Schemas

33.1 Capability card schema excerpt

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$id": "https://uap.dev/schemas/capability-card.schema.json",
  "type": "object",
  "required": [
    "capability_id",
    "name",
    "kind",
    "provider",
    "input_schema_ref",
    "output_schema_ref",
    "risk"
  ],
  "properties": {
    "capability_id": {"type": "string"},
```

```
    "name": {"type": "string"},
    "kind": {
      "type": "string",
      "enum": ["read", "write", "workflow", "agent", "transform"]
    },
    "provider": {
      "type": "object",
      "required": ["type", "provider_id"],
      "properties": {
        "type": {
          "type": "string",
          "enum": [
            "mcp",
            "openapi",
            "graphql",
            "grpc",
            "a2a",
            "local",
            "workflow",
            "asyncapi"
          ]
        },
        "provider_id": {"type": "string"}
      }
    },
    "risk": {
      "type": "object",
      "required": ["level", "side_effect"],
      "properties": {
        "level": {
          "type": "string",
          "enum": ["none", "low", "medium", "high", "critical"]
        },
        "side_effect": {"type": "boolean"},
        "data_classes": {
          "type": "array",
          "items": {"type": "string"}
        }
      }
    }
  }
}
```

33.2 Error schema excerpt

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$id": "https://uap.dev/schemas/error.schema.json",
  "type": "object",
  "required": ["code", "category", "recoverability", "safe_retry"],
  "properties": {
    "code": {"type": "string"},
    "message": {"type": "string"},
    "category": {
      "type": "string",
      "enum": [
        "invalid_input",
```

```
        "schema_mismatch",
        "auth_required",
        "consent_required",
        "policy_denied",
        "approval_required",
        "rate_limited",
        "timeout",
        "provider_unavailable",
        "context_budget_exceeded",
        "cost_budget_exceeded",
        "unsafe_output",
        "conflict",
        "unknown"
      ]
    },
    "recoverability": {
      "type": "string",
      "enum": [
        "retryable",
        "repairable_by_model",
        "repairable_by_user",
        "requires_approval",
        "requires_auth",
        "fallback_available",
        "terminal"
      ]
    },
    "safe_retry": {"type": "boolean"},
    "retry_after_ms": {"type": "integer", "minimum": 0}
  }
}
```

34 Appendix B: Open questions

1. Should UAP standardize a plan language or only the graph schema?
2. How much planning should be model-visible versus runtime-private?
3. What is the minimum useful capability attestation format?
4. Should capability embeddings be standardized or implementation-specific?
5. Should UAP require CloudEvents compatibility or only recommend it?
6. Which policy engine should be used in the reference implementation?
7. How should UAP represent mutable shared state without duplicating AG-UI?
8. How should confidential computing or sandbox attestation be represented?
9. How should billing and cost estimates be standardized?
10. What should be the first official benchmark suite?

35 Appendix C: Reference list

[R1] Model Context Protocol, “Lifecycle,” latest specification, 2025-11-25. https://modelcontextprotocol.io/specification/2025-11-25/basic/lifecycle

[R2] Model Context Protocol, specification repository and schema. https://github.com/modelcontextprotocol/modelcontextproto

[R3] Agent2Agent Protocol, “A2A Protocol Specification,” latest released version 1.0.0. https://a2a-protocol.org/latest/specificati

[R4] AG-UI, “Agent User Interaction Protocol” overview. https://docs.ag-ui.com/introduction

[R5] Agent Communication Protocol documentation. https://agentcommunicationprotocol.dev/introduction/welcome

[R6] IBM Research, "Agent Communication Protocol," noting ACP is now part of A2A under the Linux Foundation. https://research.ibm.com/projects/agent-communication-protocol

[R7] OpenAPI Specification 3.1.0. https://spec.openapis.org/oas/v3.1.0.html

[R8] AsyncAPI Specification 3.1.0. https://www.asyncapi.com/docs/reference/specification/v3.1.0

[R9] CNCF CloudEvents project. https://www.cncf.io/projects/cloudevents/

[R10] W3C Trace Context Recommendation. https://www.w3.org/TR/trace-context/

[R11] OpenTelemetry, "Context propagation." https://opentelemetry.io/docs/concepts/context-propagation/

[R12] IETF OAuth 2.1 draft. https://datatracker.ietf.org/doc/draft-ietf-oauth-v2-1/

[R13] RFC 9449, "OAuth 2.0 Demonstrating Proof of Possession (DPoP)." https://datatracker.ietf.org/doc/html/rfc9449

[R14] W3C, "Decentralized Identifiers (DIDs) v1.0 becomes a W3C Recommendation." https://www.w3.org/press-releases/2022/did-rec/

[R15] Srinivasan, V., "Bridging Protocol and Production: Design Patterns for Deploying AI Agents with Model Context Protocol," arXiv:2603.13417, 2026. https://arxiv.org/abs/2603.13417

[R16] Huang, C. et al., "Model Context Protocol Threat Modeling and Analyzing Vulnerabilities to Prompt Injection with Tool Poisoning," arXiv:2603.22489, 2026. https://arxiv.org/abs/2603.22489

[R17] Wang, Z. et al., "MCPTox: A Benchmark for Tool Poisoning Attack on Real-World MCP Servers," arXiv:2508.14925, 2025. https://arxiv.org/abs/2508.14925

[R18] Errico, H. et al., "Securing the Model Context Protocol (MCP): Risks, Controls, and Governance," arXiv:2511.20920, 2025. https://arxiv.org/abs/2511.20920

[R19] Fan, S. et al., "MCPToolBench++: A Large Scale AI Agent Model Context Protocol Tool Use Benchmark," arXiv:2508.07575, 2025. https://arxiv.org/pdf/2508.07575

[R20] Bandi, C. et al., "MCP-Atlas: A Large-Scale Benchmark for Tool-Use Competency with Real MCP Servers," arXiv:2602.00933, 2026. https://arxiv.org/abs/2602.00933

[R21] Biswas, A. et al., "SUTRADHARA: An Intelligent Orchestrator-Engine Co-design for Tool-based Agentic Inference," Microsoft Research, 2026. https://www.microsoft.com/en-us/research/publication/sutradhara-an-intelligent-orchestrator-engine-co-design-for-tool-based-agentic-inference/

[R22] Ehtesham, A. et al., "A survey of agent interoperability protocols: MCP, ACP, A2A, and ANP," arXiv:2505.02279, 2025. https://arxiv.org/abs/2505.02279

[R23] Zheng, S. and Zhang, Q., "AgentRFC: Security Design Principles and Conformance Testing for Agent Protocols," arXiv:2603.23801, 2026. https://arxiv.org/abs/2603.23801